Week 2 Day 2 — Shell and Filesystem Tools (Oct 15–16, 2026)

What was built

`shell_tools.py`: the agent's hands for the local machine — two tools, both fenced into a sandbox.

`run_command(command, timeout=30) -> dict` runs a shell command and returns its output or a structured error. The guards, in order: the command must be a non-empty list (a string gets `bad_command` — this is the no-shell rule, no `shell=True` anywhere); execution is pinned to `SAFE_DIR` via `cwd`; a timeout kills hung commands (`timeout`); a missing program returns `not_found` instead of raising. Stdout and stderr are each truncated at `MAX_OUTPUT_BYTES` so a chatty command can't flood the context window. A nonzero exit code still returns `ok: True` — the tool worked; the failed command is information for the loop.

`read_file(rel_path) -> dict` reads a text file inside the sandbox or refuses. The fence is two moves: `Path(SAFE_DIR).resolve()` canonicalizes the sandbox address once at import, then every requested path is resolved against it and checked with `is_relative_to` — anything outside comes back as `path_escape`, with both the requested and the resolved path in the report so you can see where it tried to go. The read itself is a try/except: missing file is `not_found`, undecodable bytes or an OS refusal is `decode_error`. Large files truncate at `MAX_OUTPUT_BYTES` with the flag set. Success returns the resolved path, the text, and the truncation flag.

`tools.py`: two new registry entries, `run_command` and `read_file`, alongside `fetch_url`. `run_tool` didn't change — it already routes by name.

Done criteria (all met): all three tools run through the registry on happy paths · each tool has a tested failure mode — a 404-shaped fetch failure, a forced timeout (2 seconds against a 9-second ping), and a path escape — all returned as structured observations, never exceptions. Committed as `be21fba` and pushed.

What actually happened

The method changed on Oct 15, and it stuck. First session typing the code instead of copying it, in bite-size chunks, comments in my own words. The decision at the end of the day: this is the method going forward, even if the sprint takes longer. The rest of this summary is the evidence for why.

The raw-string bug. First version: `SAFE_DIR = 'D:\agent-sprint\...'`. Python reads `\a` as the bell escape character, so the path corrupts the moment the file loads — no error, no traceback, just a wrong directory three steps later. Fix: raw string. Rule earned: Windows paths in Python are raw strings, always. Silent corruption is worse than a crash.

Two typo bugs, both on unhappy paths. `"error:"` with a colon inside the key string in the guard clause — would have KeyError'd on the one path meant to prevent crashes. `MAX_OUTPUTBYTES` missing its underscores in the truncation line — NameError the first time output exceeded the cap. The happy-path test passed clean and would never have found either one.

PowerShell is not Python, again. Typed an import at the `PS>` prompt, got a ParserError. The prompt tells you who's listening: `PS>` is PowerShell, `>>>` is Python. And `exit()` (or Ctrl+Z) gets you back out.

`dir` **is not a real program.** On Windows it's a cmd.exe builtin, so `["dir", "/w"]` raises FileNotFoundError and looks exactly like a broken tool. Test with real programs.

Git hygiene. The pull said "already up to date" — the aborted pull from the day before was the VM clone, not the laptop. But the recovery steps assumed an untracked file and `rm`'d the tracked `day-01-summary.md`; restored with `git restore`, confirmed identical with `fc.exe` (note the `.exe` — bare `fc` in PowerShell is the `Format-Custom` alias, the same species of shadowing as the `dir` gotcha). A stray whitespace-only change to Week 1's `agent.py` got reverted before committing so Day 2 stayed clean.

`read_file` **on Oct 16, typed in three bites.** Bite 1: imports and the fence. Bite 2: the try/except read. Bite 3: truncation and the success return. Two bugs caught while typing: a missing closing paren on the `def` line (the file wouldn't even load), and a comma instead of a colon in the `decode_error` return — on the unhappy path, again. A namespace lesson along the way: `from tools import run_tool` doesn't bring `run_command` into your session; each import only names what you asked for.

Comments got reviewed before the tests ran. The verdict: the why-comments are the strong ones ("preventing directory traversal attacks," "so a huge file can't flood the context window"); two typos red-penned.

The core insight

Three sessions in a row, every bug lived on an unhappy path. The happy path is the code you watch; the failure paths are the code that runs when it matters, which is exactly why they're the code nobody exercises. Typing catches these at the keyboard — both of today's bugs were found while typing, not while testing. And the fence itself turned out to be load-bearing beyond the exercise: resolve-then-`is_relative_to` is the same two-move combination Google shipped to fix a real path traversal in their agent framework. The homework guard is the industry's guard.

Line-by-line walkthrough

```
SAFE_DIR = r'D:\agent-sprint\week-02-tools-structured-output\sandbox'
```

The `r` prefix is the whole lesson: without it, `\a` is the bell character and the sandbox address is corrupt from line one. Raw strings for Windows paths, no exceptions.

```
if not isinstance(command, list) or len(command) == 0:
    return {"ok": False, "error": "bad_command", ...}
```

The no-shell rule as a type check. A string command would go through shell interpretation; a list can't. The guard makes the security decision structural instead of advisory.

```
target = (_SAFE_DIR / rel_path).resolve()
if not target.is_relative_to(_SAFE_DIR):
    return {"ok": False, "error": "path_escape", ...}
```

The fence. Resolve the request to its canonical form, then ask the one question: is it inside the sandbox? `../tools.py` resolves one level up, the check fails, and the tool reports the escape instead of reading the file. Both sides canonical, or the comparison means nothing.

```
    except (UnicodeDecodeError, OSError):
        return {"ok": False, "error": "decode_error", "requested": rel_path}
```

Two failure modes, one handler: bytes that aren't text, and the OS saying no. The contract holds — the tool returns a dict, never raises, and the loop gets an observation it can reason about.